

1. Remarks previous sessions	1
1.1. Rust tooling	2
1.2. Important exercises	3
2. Threads	4
3. Channels	12
4. Send and Sync	17
5. Shared state	25
6. Exercises	32

1.1. Rust tooling

1. Remarks previous sessions

Open the standard library docs:

```
1 rustup doc --std
```

(playground link)

1.1. Rust tooling

Open the standard library docs:

```
1 rustup doc --std
```

(playground link)

Pinning Rust version in rust-toolchain.toml:

```
1 [toolchain]
2 channel = "nightly-2025-12-08"
3 components = ["rustfmt", "clippy", "rust-analyzer"]
```

(playground link)

1.1. Rust tooling

Open the standard library docs:

```
1 rustup doc --std
```

(playground link)

Pinning Rust version in rust-toolchain.toml:

```
1 [toolchain]
2 channel = "nightly-2025-12-08"
3 components = ["rustfmt", "clippy", "rust-analyzer"]
```

(playground link)

Installing nightly Rust binaries from crates.io:

```
1 rustup update nightly
2 rustup toolchain install nightly --component rustc-dev rust-src llvm-tools
3 cargo +nightly install ferrous-owl
```

(playground link)

Add ~/.cargo/bin to your PATH if not already done.

Upgrading: cargo +nightly install ferrous-owl.

1.2. Important exercises

Make sure you have already finished:

- `session-5/tests/error-handling.rs`

1. Remarks previous sessions

1. Remarks previous sessions	1
2. Threads	4
2.1. Plain Threads	5
2.2. Receiving data from threads	7
2.3. Handling thread panics	8
2.4. Borrowing and threads	9
2.5. Scoped threads	10
2.6. Moving to threads	11
3. Channels	12
4. Send and Sync	17
5. Shared state	25
6. Exercises	32

2.1. Plain Threads

```
1  use std::thread;
2  use std::time::Duration;
3
4  fn main() {
5      thread::spawn(|| {
6          for i in 0..10 {
7              println!("Count in thread: {i}!");
8              thread::sleep(Duration::from_millis(5));
9          }
10     });
11
12     for i in 0..5 {
13         println!("Main thread: {i}");
14         thread::sleep(Duration::from_millis(5));
15     }
16 }
```

(playground link)

What happens to the spawned thread?

2.1. Plain Threads

```
1 use std::thread;
2 use std::time::Duration;
3
4 fn main() {
5     thread::spawn(|| {
6         for i in 0..10 {
7             println!("Count in thread: {i}!");
8             thread::sleep(Duration::from_millis(5));
9         }
10    });
11
12    for i in 0..5 {
13        println!("Main thread: {i}");
14        thread::sleep(Duration::from_millis(5));
15    }
16 }
```

(playground link)

What happens to the spawned thread?

Terminated by operating system when main thread exits. No destructors called.

2.1. Plain Threads

```
1 thread::spawn(|| {  
2     for i in 0..10 {  
3     });  
4  
5 for i in 0..5 {  
6     println!("Main thread: {i}");  
7     thread::sleep(Duration::from_millis(5));  
8 }
```

([playground link](#))

How can we ensure the spawned thread finishes?

2.1. Plain Threads

```
1 thread::spawn(|| {  
2     for i in 0..10 {  
3     });  
4  
5 for i in 0..5 {  
6     println!("Main thread: {i}");  
7     thread::sleep(Duration::from_millis(5));  
8 }
```

(playground link)

How can we ensure the spawned thread finishes?

Use `JoinHandle` returned by `thread::spawn` and call `join()` on it.

```
1 fn main() {  
2     let handle = thread::spawn(|| {  
3         // thread code  
4     });  
5     // main thread code  
6     handle.join();  
7 }
```

(playground link)

2.2. Receiving data from threads

How can we get a return value from a thread?

2.2. Receiving data from threads

How can we get a return value from a thread?

The `JoinHandle` returns a `Result` with the thread's return value.

```
1 fn main() {  
2     let handle = thread::spawn(|| {  
3         42  
4     });  
5  
6     let result = handle.join().unwrap();  
7     println!("The answer is: {}", result);  
8 }
```

(playground link)

What happens to the main thread if a spawned thread panics?

2.2. Receiving data from threads

How can we get a return value from a thread?

The `JoinHandle` returns a `Result` with the thread's return value.

```
1 fn main() {  
2     let handle = thread::spawn(|| {  
3         42  
4     });  
5  
6     let result = handle.join().unwrap();  
7     println!("The answer is: {}", result);  
8 }
```

(playground link)

What happens to the main thread if a spawned thread panics?

Nothing, the main thread continues executing.

2.3. Handling thread panics

```
1  fn main() {  
2      let handle = thread::spawn(|| {  
3          panic!("Thread panicked!");  
4      });  
5      let result = handle.join();  
6      match result {  
7          Ok(_) => println!("Thread completed successfully."),  
8          Err(e) => println!("Thread panicked: {:?}", e),  
9      }  
10 }
```

(playground link)

Exercise:

- examples/threads1.rs
- examples/threads2.rs

2.4. Borrowing and threads

Sending references to threads:

```
1 fn main() {  
2     let data = vec![1, 2, 3];  
3     let handle = thread::spawn(|| {  
4         println!("Data: {:?}", data);  
5     });  
6 }
```

([playground link](#))

What happens if you want a thread to borrow data from the main thread?

2.4. Borrowing and threads

Sending references to threads:

```
1 fn main() {  
2     let data = vec![1, 2, 3];  
3     let handle = thread::spawn(|| {  
4         println!("Data: {:?}", data);  
5     });  
6 }
```

([playground link](#))

What happens if you want a thread to borrow data from the main thread?

Compilation error: data may not live long enough. Use move keyword to transfer ownership.

2.4. Borrowing and threads

Sending references to threads:

```
1 fn main() {  
2     let data = vec![1, 2, 3];  
3     let handle = thread::spawn(|| {  
4         println!("Data: {:?}", data);  
5     });  
6 }
```

[\(playground link\)](#)

What happens if you want a thread to borrow data from the main thread?

Compilation error: data may not live long enough. Use move keyword to transfer ownership.

```
1 fn main() {  
2     let data = vec![1, 2, 3];  
3     let handle = thread::spawn(|| {  
4         println!("Data: {:?}", data);  
5     });  
6     handle.join().unwrap();  
7 }
```

[\(playground link\)](#)

What happens if you add a join?

2.4. Borrowing and threads

Sending references to threads:

```
1 fn main() {  
2     let data = vec![1, 2, 3];  
3     let handle = thread::spawn(|| {  
4         println!("Data: {:?}", data);  
5     });  
6 }
```

[\(playground link\)](#)

What happens if you want a thread to borrow data from the main thread?

Compilation error: data may not live long enough. Use move keyword to transfer ownership.

```
1 fn main() {  
2     let data = vec![1, 2, 3];  
3     let handle = thread::spawn(|| {  
4         println!("Data: {:?}", data);  
5     });  
6     handle.join().unwrap();  
7 }
```

[\(playground link\)](#)

What happens if you add a join?

Same error. Rust cannot verify join is called before data goes out of scope (at compile time).

2.5. Scoped threads

```
1 fn main() {  
2     let data = vec![1, 2, 3];  
3     let handle = thread::spawn(|| {  
4         println!("Data: {:?}", data);  
5     });  
6     handle.join().unwrap();  
7 }
```

([playground link](#))

2.5. Scoped threads

```
1 fn main() {  
2     let data = vec![1, 2, 3];  
3     let handle = thread::spawn(|| {  
4         println!("Data: {:?}", data);  
5     });  
6     handle.join().unwrap();  
7 }
```

(playground link)

Solution: `thread::scope` guarantees all spawned threads complete before the scope ends.

```
1 fn main() {  
2     let data = vec![1, 2, 3];  
3     thread::scope(|s| {  
4         s.spawn(|| {  
5             println!("Data: {:?}", data);  
6         });  
7     });  
8 }
```

(playground link)

Why can scoped threads borrow data?

2.5. Scoped threads

```
1 fn main() {  
2     let data = vec![1, 2, 3];  
3     let handle = thread::spawn(|| {  
4         println!("Data: {:?}", data);  
5     });  
6     handle.join().unwrap();  
7 }
```

(playground link)

Solution: `thread::scope` guarantees all spawned threads complete before the scope ends.

```
1 fn main() {  
2     let data = vec![1, 2, 3];  
3     thread::scope(|s| {  
4         s.spawn(|| {  
5             println!("Data: {:?}", data);  
6         });  
7     });  
8 }
```

(playground link)

Why can scoped threads borrow data?

The scope blocks until all threads finish, so the compiler knows data outlives the threads.

2.6. Moving to threads

You can also send input to spawned threads.

- using a channel sender (see next section)
- using the `move` keyword to transfer ownership at spawn time

Since spawn time means **at runtime**, non-`'static` references cannot be moved to threads.

```
1 fn spawn<F, T>(f: F) -> JoinHandle<T>
2 where
3     F: FnOnce() -> T + Send + 'static,
4     T: Send + 'static
```

([playground link](#))

2.6. Moving to threads

You can also send input to spawned threads.

- using a channel sender (see next section)
- using the `move` keyword to transfer ownership at spawn time

Since spawn time means **at runtime**, non-`'static` references cannot be moved to threads.

```
1 fn spawn<F, T>(f: F) -> JoinHandle<T>
2 where
3     F: FnOnce() -> T + Send + 'static,
4     T: Send + 'static
```

[\(playground link\)](#)

```
1 fn main() {
2     let data = String::from("hello");
3     let handle = thread::spawn(move || {
4         println!("Data: {data}"); // `data` moved into closure, now owned by thread
5     });
6     // println!("{data}"); // Error: value moved
7     handle.join().unwrap();
8 }
```

} (1) closure f must be 'static
[\(playground link\)](#)

Why does `thread::spawn` require `'static`?

2.6. Moving to threads

You can also send input to spawned threads.

- using a channel sender (see next section)
- using the `move` keyword to transfer ownership at spawn time

Since spawn time means **at runtime**, non-`'static` references cannot be moved to threads.

```
1 fn spawn<F, T>(f: F) -> JoinHandle<T>
2 where
3     F: FnOnce() -> T + Send + 'static,
4     T: Send + 'static
```

[\(playground link\)](#)

```
1 fn main() {
2     let data = String::from("hello");
3     let handle = thread::spawn(move || {
4         println!("Data: {data}"); // `data` moved into closure, now owned by thread
5     });
6     // println!("{data}"); // Error: value moved
7     handle.join().unwrap();
8 }
```

} (1) closure f must be 'static
[\(playground link\)](#)

Why does `thread::spawn` require `'static`?

The spawned thread may outlive the caller. Without `'static`, the closure could hold dangling references.

1. Remarks previous sessions	1
2. Threads	4
3. Channels	12
3.1. Senders and Receivers	13
3.2. Unbounded Channels	14
3.3. Bounded channels	15
3.4. Exercises	16
4. Send and Sync	17
5. Shared state	25
6. Exercises	32

What does `mpsc` stand for?

3.1. Senders and Receivers

What does mpsc stand for?

Multiple Producer, Single Consumer.

```
1  fn main() {  
2      let (tx, rx) = std::sync::mpsc::channel();  
3  
4      tx.send(10).unwrap();  
5      tx.send(20).unwrap();  
6  
7      println!("Received: {:?}", rx.recv());  
8      println!("Received: {:?}", rx.recv());  
9  
10     let tx2 = tx.clone();  
11     tx2.send(30).unwrap();  
12     println!("Received: {:?}", rx.recv());  
13 }
```

(playground link)

What happens when you call send and all receivers have been dropped?

3.1. Senders and Receivers

What does mpsc stand for?

Multiple Producer, Single Consumer.

```
1  fn main() {  
2      let (tx, rx) = std::sync::mpsc::channel();  
3  
4      tx.send(10).unwrap();  
5      tx.send(20).unwrap();  
6  
7      println!("Received: {:?}", rx.recv());  
8      println!("Received: {:?}", rx.recv());  
9  
10     let tx2 = tx.clone();  
11     tx2.send(30).unwrap();  
12     println!("Received: {:?}", rx.recv());  
13 }
```

(playground link)

What happens when you call send and all receivers have been dropped?

send() returns an error (channel is “closed”).

What happens when you call send and no receivers are listening?

3.2. Unbounded Channels

What happens when you call send and no receivers are listening?

Message is queued until a receiver is available.

```
1  fn main() {
2      let (tx, rx) = std::sync::mpsc::channel();
3
4      std::thread::spawn(move || {
5          let thread_id = std::thread::current().id();
6          for i in 0..10 {
7              tx.send(format!("Message {i}")).unwrap();
8              println!("{thread_id:?}: sent Message {i}");
9          }
10         println!("{thread_id:?}: done");
11     });
12     std::thread::sleep(std::time::Duration::from_millis(100));
13
14     for msg in rx.iter() {
15         println!("Main: got {msg}");
16     }
17 }
```

[\(playground link\)](#)

3.3. Bounded channels

```
1  use std::sync::mpsc;
2  use std::thread;
3  use std::time::Duration;
4
5  fn main() {
6      let (tx, rx) = mpsc::sync_channel(3);
7
8      thread::spawn(move || {
9          let thread_id = thread::current().id();
10         for i in 0..10 {
11             tx.send(format!("Message {i}")).unwrap();
12             println!("{thread_id:?}: sent Message {i}");
13         }
14         println!("{thread_id:?}: done");
15     });
16     thread::sleep(Duration::from_millis(100));
17
18     for msg in rx.iter() {
19         println!("Main: got {msg}");
20     }
21 }
```

[\(playground link\)](#)

3.4. Exercises

- `examples/threads3.rs`

1. Remarks previous sessions	1
2. Threads	4
3. Channels	12
4. Send and Sync	17
4.1. Marker Traits	18
4.2. Send	19
4.3. Sync	20
4.4. Examples	21
4.5. Summary	24
5. Shared state	25
6. Exercises	32

4.1. Marker Traits

How does Rust know to forbid shared access across threads?

4.1. Marker Traits

How does Rust know to forbid shared access across threads?

- Send: a type T is Send if it is safe to move a T across a thread boundary.

4.1. Marker Traits

How does Rust know to forbid shared access across threads?

- Send: a type T is Send if it is safe to move a T across a thread boundary.
- Sync: a type T is Sync if it is safe to move a &T across a thread boundary.

4.1. Marker Traits

How does Rust know to forbid shared access across threads?

- Send: a type T is Send if it is safe to move a T across a thread boundary.
- Sync: a type T is Sync if it is safe to move a &T across a thread boundary.

Info

Send and Sync are unsafe auto traits.

What are auto traits?

4.1. Marker Traits

How does Rust know to forbid shared access across threads?

- Send: a type T is Send if it is safe to move a T across a thread boundary.
- Sync: a type T is Sync if it is safe to move a &T across a thread boundary.

Info

Send and Sync are unsafe auto traits.

What are auto traits?

The compiler will automatically derive them for your types as long as they only contain types that implement the auto trait.

What are unsafe traits?

4.1. Marker Traits

How does Rust know to forbid shared access across threads?

- Send: a type T is Send if it is safe to move a T across a thread boundary.
- Sync: a type T is Sync if it is safe to move a &T across a thread boundary.

Info

Send and Sync are unsafe auto traits.

What are auto traits?

The compiler will automatically derive them for your types as long as they only contain types that implement the auto trait.

What are unsafe traits?

Traits that have safety invariants that the compiler cannot verify automatically. Implementing them incorrectly can lead to undefined behavior.

You can implement unsafe traits manually when you know it is valid.

4.1. Marker Traits

How does Rust know to forbid shared access across threads?

- Send: a type T is Send if it is safe to move a T across a thread boundary.
- Sync: a type T is Sync if it is safe to move a &T across a thread boundary.

Info

Send and Sync are unsafe auto traits.

What are auto traits?

The compiler will automatically derive them for your types as long as they only contain types that implement the auto trait.

What are unsafe traits?

Traits that have safety invariants that the compiler cannot verify automatically. Implementing them incorrectly can lead to undefined behavior.

You can implement unsafe traits manually when you know it is valid.

Warning

Implementing unsafe traits requires unsafe blocks. Using them as constraints does not.

4.2. Send

A type `T` is `Send` if it is **safe to move a `T` value to another thread**.

4.2. Send

A type `T` is `Send` if it is **safe to move a `T` value to another thread**.

Moving ownership entails:

- running destructor in the other thread
- accessing the value in the other thread
- taking mutable references to the value in the other thread

4.2. Send

A type T is Send if it is **safe to move a T value to another thread**.

Moving ownership entails:

- running destructor in the other thread
- accessing the value in the other thread
- taking mutable references to the value in the other thread

```
1  use std::thread;
2
3  fn main() {
4      let data = vec![1, 2, 3]; // Vec<i32> is Send
5
6      let handle = thread::spawn(move || {
7          println!("Data in thread: {:?}", data);
8          data.iter().sum::<i32>()
9      });
10
11     let result = handle.join().unwrap();
12     println!("Sum: {}", result);
13 }
```

[\(playground link\)](#)

This compiles because Vec<i32> implements Send.

4.3. Sync

A value of type `T` is `Sync` if and only if an immutable reference to `T` (`&T`) is `Send`.

In practice this means that it is safe to **access immutably from several threads in parallel**.

Corollary 4.3.0.1. *A consequence of this is that if `T: !Sync` then `&T: !Send`.*

4.3. Sync

A value of type T is Sync if and only if an immutable reference to T ($\&T$) is Send.

In practice this means that it is safe to **access immutably from several threads in parallel**.

Corollary 4.3.0.2. *A consequence of this is that if $T: !\text{Sync}$ then $\&T: !\text{Send}$.*

Proof.

$$\&T \Leftrightarrow \&\&T$$

$$T : \text{Sync} \Leftrightarrow \&T : \text{Send}$$



4.4. Examples

4.4.1. Send + Sync

Most types you come across are Send + Sync:

- **Owned data, no interior mutability:** i32, bool, String, Vec<T>

4.4. Examples

4.4.1. Send + Sync

Most types you come across are Send + Sync:

- **Owned data, no interior mutability:** `i32`, `bool`, `String`, `Vec<T>`
- **Thread-safe shared ownership:** `Arc<T>` (atomic ref counting)

4.4. Examples

4.4.1. Send + Sync

Most types you come across are Send + Sync:

- **Owned data, no interior mutability:** `i32`, `bool`, `String`, `Vec<T>`
- **Thread-safe shared ownership:** `Arc<T>` (atomic ref counting)
- **Hardware-supported atomics:** `AtomicBool`, `AtomicU8` (compare-and-swap, fetch-and-add)

When are the generic types such as `Vec<T>` Send + Sync

4.4. Examples

4.4.1. Send + Sync

Most types you come across are Send + Sync:

- **Owned data, no interior mutability:** `i32`, `bool`, `String`, `Vec<T>`
- **Thread-safe shared ownership:** `Arc<T>` (atomic ref counting)
- **Hardware-supported atomics:** `AtomicBool`, `AtomicU8` (compare-and-swap, fetch-and-add)

When are the generic types such as `Vec<T>` Send + Sync

When the type parameters `T` are Send + Sync.

4.4.2. Send + !Sync

These types can be moved to other threads, but cannot be shared via `&T`:

4.4. Examples

4.4.1. Send + Sync

Most types you come across are Send + Sync:

- **Owned data, no interior mutability:** `i32`, `bool`, `String`, `Vec<T>`
- **Thread-safe shared ownership:** `Arc<T>` (atomic ref counting)
- **Hardware-supported atomics:** `AtomicBool`, `AtomicU8` (compare-and-swap, fetch-and-add)

When are the generic types such as `Vec<T>` Send + Sync

When the type parameters `T` are Send + Sync.

4.4.2. Send + !Sync

These types can be moved to other threads, but cannot be shared via `&T`:

- **Non-thread-safe interior mutability:** `Cell<T>`, `RefCell<T>`
 - Allow mutation through `&T` without synchronization
 - Safe to own exclusively on one thread, unsafe to share references

4.4. Examples

4.4.3. !Send + Sync

These types are safe to access (via shared references) from multiple threads, but cannot be moved to another thread:

- **Lock guards:** `MutexGuard`, `RwLockReadGuard`, `RwLockWriteGuard`
 - Must be unlocked on the thread that acquired the lock (POSIX requirement)

Proposition 4.4.1. *`MutexGuard<T: Sync>: !Send + Sync`*

4.4. Examples

4.4.3. !Send + Sync

These types are safe to access (via shared references) from multiple threads, but cannot be moved to another thread:

- **Lock guards:** `MutexGuard`, `RwLockReadGuard`, `RwLockWriteGuard`
 - Must be unlocked on the thread that acquired the lock (POSIX requirement)

Proposition 4.4.1. $\text{MutexGuard}<T: \text{Sync}>: !\text{Send} + \text{Sync}$

Proof. **Part 1:** $\text{MutexGuard}<T: \text{Sync}>: !\text{Send}$

$$\begin{aligned} \text{drop}(\text{MutexGuard}) &\Rightarrow \text{Mutex}::\text{unlock}() \\ \text{Mutex}::\text{unlock}() &\Rightarrow \text{must run on locking thread} \\ \text{MutexGuard} : \text{Send} &\Rightarrow \text{drop can run on any thread} \\ \therefore \text{MutexGuard} &: !\text{Send} \end{aligned}$$

Part 2: $\text{MutexGuard}<T: \text{Sync}>: \text{Sync}$

$$\begin{aligned} \text{MutexGuard}<T> &: \text{Sync} \\ \Leftrightarrow \&\text{MutexGuard}<T> : \text{Send} \\ &\Leftrightarrow \&T : \text{Send} \quad (\text{deref coercion}) \\ &\Leftrightarrow T : \text{Sync} \quad \checkmark \end{aligned}$$

□

4.4. Examples

4.4.4. !Send + !Sync

These types are not thread-safe and cannot be moved to other threads:

4.4. Examples

4.4.4. !Send + !Sync

These types are not thread-safe and cannot be moved to other threads:

- dereferences and has non-atomic count: `Rc<T>`
- considered unsafe in general `*const T`, `*mut T`

	Sync	!Sync
Send	i32, bool, String, Vec<T> Arc<T>, Mutex<T>, Atomic* Safe to move and share	Cell<T>, RefCell<T> Interior mutability: safe to move, not share
!Send	MutexGuard, RwLock*Guard Thread-bound: safe to share, not move	Rc<T>, *const T, *mut T Not thread-safe at all

1. Remarks previous sessions	1
2. Threads	4
3. Channels	12
4. Send and Sync	17
5. Shared state	25
5.1. Arc	26
5.2. Mutex	28
5.3. Interior mutability	30
5.4. Example	31
6. Exercises	32

5.1. Arc

```
1  use std::sync::Arc; use std::thread;
2  #[derive(Debug)]
3  struct WhereDropped(Vec<i32>); /// A struct that prints which thread drops it.
4  impl Drop for WhereDropped {
5      fn drop(&mut self) { println!("Dropped by {:?}", thread::current().id()) }
6  }
7  fn main() {
8      let v = Arc::new(WhereDropped(vec![10, 20, 30]));
9      let mut handles = Vec::new();
10     for i in 0..5 {
11         let v = Arc::clone(&v);
12         handles.push(thread::spawn(move || {
13             // Sleep for 0-500ms.
14             std::thread::sleep(std::time::Duration::from_millis(500 - i * 100));
15             let thread_id = thread::current().id();
16             println!("{thread_id:?}: {v:?}");
17         }));
18     }
19     drop(v); // Now only the spawned threads will hold clones of `v`.
20     // When the last spawned thread finishes, it will drop `v`'s contents.
21     handles.into_iter().for_each(|h| h.join().unwrap());
22 }
```

[\(playground link\)](#)

What does `Arc<T>` stand for?

5.1. Arc

What does Arc<T> stand for?

Atomic Reference Counted.

Atomic refers to the fact that the **reference count is updated using atomic operations**, making atomics (AtomicUsize) and Arc<T> safe to share between threads.

When does Arc<T> implement Clone?

5.1. Arc

What does Arc<T> stand for?

Atomic Reference Counted.

Atomic refers to the fact that the **reference count is updated using atomic operations**, making atomics (AtomicUsize) and Arc<T> safe to share between threads.

When does Arc<T> implement Clone?

Always. Cloning an Arc only increments the reference count.

When does Arc<T> implement Send and Sync?

5.1. Arc

What does Arc<T> stand for?

Atomic Reference Counted.

Atomic refers to the fact that the **reference count is updated using atomic operations**, making atomics (AtomicUsize) and Arc<T> safe to share between threads.

When does Arc<T> implement Clone?

Always. Cloning an Arc only increments the reference count.

When does Arc<T> implement Send and Sync?

When T implements Send and Sync.

5.2. Mutex

Mutex allows mutable access to T behind a read-only interface:

Get `&mut T` from an `&Mutex<T>` by taking the lock.

```
1  use std::sync::Mutex;
2
3  fn main() {
4      let v = Mutex::new(vec![10, 20, 30]);
5      println!("v: {:?}", v.lock().unwrap());
6
7      {
8          let mut guard = v.lock().unwrap();
9          guard.push(40);
10     }
11
12     println!("v: {:?}", v.lock().unwrap());
13 }
```

[\(playground link\)](#)

The `lock()` returns a `Result<MutexGuard<T>, PoisonError>`. What is a `PoisonError`?

5.2. Mutex

Mutex allows mutable access to T behind a read-only interface:

Get `&mut T` from an `&Mutex<T>` by taking the lock.

```
1  use std::sync::Mutex;
2
3  fn main() {
4      let v = Mutex::new(vec![10, 20, 30]);
5      println!("v: {:?}", v.lock().unwrap());
6
7      {
8          let mut guard = v.lock().unwrap();
9          guard.push(40);
10     }
11
12     println!("v: {:?}", v.lock().unwrap());
13 }
```

[\(playground link\)](#)

The `lock()` returns a `Result<MutexGuard<T>, PoisonError>`. What is a `PoisonError`?

It indicates that another thread panicked while holding the lock, potentially leaving the data in an inconsistent state.

Why is `Mutex<T>`: Sync when `T`: Send (not `T`: Sync)?

5.2. Mutex

Why is `Mutex<T>`: `Sync` when `T`: `Send` (not `T`: `Sync`)?

The mutex ensures exclusive access - only one thread touches `T` at a time, so `T` doesn't need to support concurrent sharing.

```
1 unsafe impl<T: ?Sized + Send> Sync for Mutex<T> {}
```

[\(playground link\)](#)

5.3. Interior mutability

Definition 5.3.1 (Interior mutability). *A type that allows mutation through a shared reference ($\&T \rightarrow \&\text{mut inner}$).*

5.3. Interior mutability

Definition 5.3.1 (Interior mutability). *A type that allows mutation through a shared reference ($\&T \rightarrow \&mut\ inner$).*

Types with interior mutability:

Type	Thread-safe?	Checking	Use case
<code>Cell<T></code>	No	Compile-time	Simple values, Copy types
<code>RefCell<T></code>	No	Runtime (panics)	Complex borrows, single-threaded
<code>Mutex<T></code>	Yes	Runtime (blocks)	Exclusive access across threads
<code>RwLock<T></code>	Yes	Runtime (blocks)	Many readers, few writers
<code>Atomic*</code>	Yes	Hardware	Counters, flags, lock-free

5.3. Interior mutability

Definition 5.3.1 (Interior mutability). *A type that allows mutation through a shared reference ($\&T \rightarrow \&mut\ inner$).*

Types with interior mutability:

Type	Thread-safe?	Checking	Use case
<code>Cell<T></code>	No	Compile-time	Simple values, Copy types
<code>RefCell<T></code>	No	Runtime (panics)	Complex borrows, single-threaded
<code>Mutex<T></code>	Yes	Runtime (blocks)	Exclusive access across threads
<code>RwLock<T></code>	Yes	Runtime (blocks)	Many readers, few writers
<code>Atomic*</code>	Yes	Hardware	Counters, flags, lock-free

5.4. Example

Open `examples/demo.rs`

5.4. Example

Open `examples/demo.rs`

Typically Rust beginners will:

1. Wrap `T` in `Mutex::new(Arc::new(T))`
2. Clone the `Arc` to share ownership across threads
3. Lock the `Mutex` to get mutable access to `T` in each thread

There are some problems with this approach:

5.4. Example

Open `examples/demo.rs`

Typically Rust beginners will:

1. Wrap `T` in `Mutex::new(Arc::new(T))`
2. Clone the `Arc` to share ownership across threads
3. Lock the `Mutex` to get mutable access to `T` in each thread

There are some problems with this approach:

- **Deadlocks**: the guard returned by `.lock()` must be dropped and a costly operation (like I/O) performed while holding the lock.

5.4. Example

Open `examples/demo.rs`

Typically Rust beginners will:

1. Wrap `T` in `Mutex::new(Arc::new(T))`
2. Clone the `Arc` to share ownership across threads
3. Lock the `Mutex` to get mutable access to `T` in each thread

There are some problems with this approach:

- **Deadlocks**: the guard returned by `.lock()` must be dropped and a costly operation (like I/O) performed while holding the lock.
- **Coarse locking**: the whole data structure is locked, even when only a small part needs to be mutated.

5.4. Example

Open `examples/demo.rs`

Typically Rust beginners will:

1. Wrap `T` in `Mutex::new(Arc::new(T))`
2. Clone the `Arc` to share ownership across threads
3. Lock the `Mutex` to get mutable access to `T` in each thread

There are some problems with this approach:

- **Deadlocks**: the guard returned by `.lock()` must be dropped and a costly operation (like I/O) performed while holding the lock.
- **Coarse locking**: the whole data structure is locked, even when only a small part needs to be mutated.
- **Global mutability**: all threads can mutate the data, making reasoning about the program harder.

5.4. Example

Open `examples/demo.rs`

Typically Rust beginners will:

1. Wrap `T` in `Mutex::new(Arc::new(T))`
2. Clone the `Arc` to share ownership across threads
3. Lock the `Mutex` to get mutable access to `T` in each thread

There are some problems with this approach:

- **Deadlocks**: the guard returned by `.lock()` must be dropped and a costly operation (like I/O) performed while holding the lock.
- **Coarse locking**: the whole data structure is locked, even when only a small part needs to be mutated.
- **Global mutability**: all threads can mutate the data, making reasoning about the program harder.

My advice:

- Minimise or remove usage of `Arc<Mutex<T>>`.
- Use at most one `Arc<Mutex<T>>` per program.
- Use `RwLock<T>` when possible.
- Use `Atomic` types for simple shared counters/flags.

1. Remarks previous sessions	1
2. Threads	4
3. Channels	12
4. Send and Sync	17
5. Shared state	25
6. Exercises	32

From Google:

- [examples/philosophers.rs](#)
- [examples/link.rs](#)